

LE ECCEZIONI

prof.ssa Taffurelli

CHE COSA SONO LE ECCEZIONI

Un'eccezione è un evento imprevisto che **causa un errore** durante l'esecuzione di un programma.

Un esempio comune è la divisione per zero, ma può verificarsi anche in altre situazioni, come la conversione di una stringa in un numero quando il formato non è valido.

Senza una gestione adeguata, un'eccezione **provoca l'interruzione improvvisa** del programma con un messaggio di errore. Per evitare ciò, è possibile utilizzare la gestione delle eccezioni, che permette di controllare e reagire agli errori in modo appropriato.

Le eccezioni vengono sollevate quando si verifica un'anomalia. Se ignorate, possono causare comportamenti inattesi o crash del programma. Tuttavia, è possibile intercettarle e gestirle per evitare problemi.

Per catturare un'eccezione si utilizza il costrutto **try...catch**

IL COSTRUTTO TRY...CATCH

Si compone di due parti

- **try** => consente di provare ad eseguire un codice che potrebbe provocare qualche eccezione

- **catch** => viene eseguita soltanto se un'eccezione viene catturata; contiene il codice per gestire l'eccezione stessa.

```
try{  
    //codice sotto controllo di eccezione  
}  
  
catch{  
    //codice eseguito in caso di  
eccezione  
}
```

esempio

```
try
{
    Console.Write("inserisci il divisore a ");
    int a = int.Parse(Console.ReadLine());
    Console.Write("inserisci il dividendo b ");
    int b = int.Parse(Console.ReadLine());
    int ris = a / b;
    Console.WriteLine($"{a}/{b}={ris}");
}
catch {
    Console.WriteLine("si è verificato un errore!");
}
```

CATCH SPECIFICO

CATCH GENERICO

L'esempio visto prima è un caso di catch generico, che permette di catturare **QUALSIASI TIPO** di eccezione si possa verificare nel try corrispondente.

Ad es sia se scriviamo 0 oppure una lettera comunque viene avviato catch

CATCH SPECIFICO

Nella clausola catch si può indicare il **tipo di eccezione** che si vuole gestire; ad esempio:

```
catch( TipoEccezione [variabile]
{
    //codice per gestire il tipo TipoEccezione
}
```

Le parentesi quadre messe attorno a variabile indicano che il nome della *variabile* stessa è opzionale e può essere anche specificato solo il tipo

esempio

Se volessimo, per es., eseguire azioni di recupero specifiche, come stampare un messaggio specifico, per il tipo di eccezione `DivideByZeroException`

```
try
{
    Console.Write("inserisci il divisore a ");
    int a = int.Parse(Console.ReadLine());
    Console.Write("inserisci il dividendo b ");
    int b = int.Parse(Console.ReadLine());
    int ris = a / b;
    Console.WriteLine("{0}/{1}={2}", a, b, ris);
}
catch(DivideByZeroException) {
    Console.WriteLine("non è possibile dividere per 0!");
}
```

LA PROPRIETA' MESSAGE

Le eccezioni sono oggetti → hanno delle proprietà, tra cui **Message**, che è una stringa di descrizione della **causa** dell'eccezione.

Questa proprietà è molto importante in fase di debug, perché lo sviluppatore può usare queste informazioni per rendersi conto dei motivi che hanno portato all'errore.

```
try
{
    Console.Write("inserisci il divisore a ");
    int a = int.Parse(Console.ReadLine());
    Console.Write("inserisci il dividendo b ");
    int b = int.Parse(Console.ReadLine());
    int ris = a / b;
    Console.WriteLine("{0}/{1}={2}", a, b, ris);
}
catch(DivideByZeroException dex) {
    Console.WriteLine(dex.Message);
}
```

CATCH MULTIPLI

All'interno di un blocco try possono essere molteplici le condizioni di errore che possono verificarsi e ognuna di esse potrebbe richiedere un trattamento diverso

E' quindi possibile utilizzare più blocchi catch, specificando differenti tipi di eccezione per ognuno di essi

Inoltre poiché non è pratico catturare TUTTI i tipi di eccezione che si possono verificare, si inserisce alla fine dell'elenco, un **catch generico**, e si demanda a questo la gestione delle eccezioni non elencate

```
try
{
    //codice sotto controllo di eccezione
}
catch( DivideByZeroException dex)
{
    Console.WriteLine(dex.Message);
}
catch (FormatException) {
    Console.WriteLine("formato non
valido !");
}
catch
{
    //qualsiasi altro tipo di eccezione
}
```


TRY...CATCH...FINALLY

Un altro costrutto fondamentale è il blocco

try...catch...finally

Il blocco **finally** **viene eseguito in qualunque caso** e viene utilizzato quando si vuol essere sicuri che una data porzione di codice venga eseguita in ogni caso.

L'unico motivo per cui un blocco **finally** non sia eseguito è che il programma termini prima che tale blocco sia eseguito

Se uno o più blocchi **catch** seguono un blocco **try**, il blocco **finally** è opzionale.

Se invece nessun blocco **catch** segue un blocco **try**, il blocco **finally** deve apparire immediatamente dopo il **try**.

```
try
{
    //codice sotto controllo di eccezione
}
catch
{
    // codice eseguito in caso di eccezione
}
finally
{
    //codice eseguito in ogni caso
}
```

L'ISTRUZIONE THROW

In alcuni casi è necessario eseguire del codice in seguito all'eccezione ma lasciare comunque al compilatore la naturale gestione della stessa. Per ottenere tale comportamento è sufficiente inserire l'istruzione **throw** all'interno del blocco catch

L'istruzione **throw** si può usare anche al di fuori del blocco catch per sollevare esplicitamente un'eccezione. In tal caso va seguita dal nome del tipo di eccezione che si vuole provocare

```
try{  
    //codice sono controllo di eccezione  
}  
catch{  
    //codice eseguito in caso di eccezione  
    throw; // risolve l'eccezione  
}
```

```
if(n<0 || n>10)  
    throw new Exception ("valore fuori dai limiti");
```

Esempio di Throw

```
static void Main(string[] args)
{
    Student std = null;
    try
    {
        PrintStudentName(std);
    }
    catch(Exception ex)
    {
        Console.WriteLine(ex.Message );
    }
    Console.ReadKey();
}

private static void PrintStudentName( Student std)
{
    if (std == null)
        throw new NullReferenceException("Student object is null.");

    Console.WriteLine(std.StudentName);
}
```

Riassumendo...

	PAROLE CHIAVE PER LA GESTIONE DELLE ECCEZIONI
TRY	Identifica un blocco di codice che può generare un'eccezione
CATCH	Identifica un blocco di codice dedicato alla gestione di tutte o di un particolare tipo di eccezione
FINALLY	Identifica un blocco di codice da eseguire a prescindere dal verificarsi o meno di un'eccezione
THROW	Istruzione utilizzata per generare una nuova eccezione, o per rilanciare un'eccezione già verificatasi rimandando la sua gestione al compilatore

https://www.w3schools.com/cs/cs_exceptions.php